

IC526 Advanced Compilers

Lec 09: Type Checker II

Yoonseung Kim

DGIST EECS

Fall 2026

Recap: Bidirectional Type Checking

$\text{check}(\Gamma, e, \tau) = \text{true} \Rightarrow \Gamma \vdash e : \tau$

$\text{synth}(\Gamma, e) = \tau \Rightarrow \Gamma \vdash e : \tau$

- These are soundness claims about the algorithm.
- The two functions are mutually recursive.

Type Checker of RustY

What To Do With a Type Checker

- The type checker will filter out invalid programs.
 - It reports an error when it cannot establish a required typing judgment.
- We will populate programs with type annotations.
 - It also fills in omitted types in let bindings.

```
let x = f(y && b) + 3;  
...
```



```
let x : i32 = (  
  (f(  
    ((y : bool) && (b : bool))  
    : bool)  
  ) : i32)  
  + (3:i32)  
  : i32);  
...
```

Why Type Annotations to All Expressions?

- Type annotations help determine which LLVM types and instructions to use.

```
%t = add i32 %a, 3 // i32
```

```
%t = add i64 %a, 3 // usize
```

```
let x =  
  (f(3) + 4) + (2 * 5);
```

```
...
```

```
%t1 = add ? %call, 4
```

```
%t = add ? %t1, %t2
```

```
let x : i32 =  
  (.. + .. : i32);
```

```
...
```

```
%t = add i32 %t1, %t2
```

Types in RustY: Design

Value Types:

$\tau ::= \text{i32} \mid \text{usize} \mid \text{bool} \mid () \mid [\tau; n]$ `let x : i32 = f(a) + 5;`

Parameter Types:

$\rho ::= \tau \mid \&\tau \mid \&\text{mut } \tau$

```
let mut arr1 : [i32; 3] = ...;
let arr2 : [i32; 3] = ...;
...
eadd(&mut arr1 : &mut [i32; 3],
    &arr2           : &[i32;3]);
```

Function Types:

$\alpha ::= \text{List}(\rho) \rightarrow \tau$

```
fn eadd(a1: &mut [i32;3], a2: &[i32;3])
{
    ..
}
```

Types in RustY: Implementation

Value types

```
type typ = T_i32 | T_usize | T_bool  
         | T_unit | T_arr of (typ * int)
```

Parameter types

```
type param_typ = PT_base of typ  
               | PT_ref of typ | PT_mutref of typ
```

Function signatures

```
type fn_info = { fn_param_types : param_typ list;  
                 fn_ret_type : typ }
```

Expressions in RustY

```
type expr = E_Lit of literal (* literal = Num of int | Bool of bool *)
           | E_Var of var      (* x *)
           | E_Deref of var     (* *x *)
           | E_Arr of expr list (* [3+1, fib(3), ..., 7] *)
           | E_Unop of unop * expr (* - g(42) *)
           | E_Binop of binop * expr * expr (* f(a && b) + (-5) *)
           | E_ArrIdx of expr * expr (* (*arr)[i+1] *)
           | E_Call of var * arg_expr list (* f(&mut x, &y, z+1) *)
```

```
and arg_expr = AE_expr of expr (* e *)
              | AE_ref of var   (* &x *)
              | AE_mutref of var (* &mut x *)
```

Typed Expressions in RustY

```
type texpr = TE_Lit of literal * typ
            | TE_Var of var * typ
            | TE_Deref of var * typ
            | TE_Arr of texpr list * typ
            | TE_Unop of unop * texpr * typ
            | TE_Binop of binop * texpr * texpr * typ
            | TE_ArrIdx of texpr * texpr * typ
            | TE_Call of var * targ_expr list * typ
```

```
and targ_expr = TAE_expr of texpr
                | TAE_ref of var
                | TAE_mutref of var
                | TAE_fwd of var
```



```
fn f(arr: &mut[i32; 3]) {
    ...
    g(arr);
    ...
}
```

Types of tc_check & tc_synth

Last Class

- $\text{check}(\Gamma : \text{env},$
 $e : \text{expr},$
 $\tau : \text{typ})$
 $: \text{bool}$
- $\text{synth}(\Gamma : \text{env},$
 $e : \text{expr})$
 $: \text{typ}$

RustY Compiler

- $\text{tc_check}(\Gamma f : \text{fenv},$
 $\Gamma v : \text{venv},$
 $e : \text{expr},$
 $\tau : \text{typ})$
 $: \text{texpr}$
- $\text{tc_synth}(\Gamma f : \text{fenv},$
 $\Gamma v : \text{venv},$
 $e : \text{expr})$
 $: \text{texpr}$

Types of `tc_check` & `tc_synth` (cont.)

`tc_check( $\Gamma f$ : fenv,  $\Gamma v$ : venv, e: expr,  $\tau$ : typ): texpr`
`tc_synth( $\Gamma f$ : fenv,  $\Gamma v$ : venv, e: expr): texpr`

- *fenv* contains **global** function signatures.

```
 $\Gamma f$  = { fib : [i32] -> i32;  
        eadd : [&mut [i32; 3], &[i32; 3]] -> unit;  
        main : [] -> unit }
```

- *venv* contains **local** binding information.

```
 $\Gamma v$  = { y : i32; x : i32; n : i32 }
```

Integer Literals: Which Type?

Synthesis chooses i32. Checking uses the expected numeric type.

```
let x = 3;           // x : i32
let y: usize = 3;    // y : usize
```

```
tc_synth( $\Gamma f$ ,  $\Gamma v$ , 3) = (3 : i32)
```

```
tc_check( $\Gamma f$ ,  $\Gamma v$ , 3, i32) = (3 : i32)
```

```
tc_check( $\Gamma f$ ,  $\Gamma v$ , 3, usize) = (3 : usize)
```

```
tc_check( $\Gamma f$ ,  $\Gamma v$ , 3, bool) raises TypeError
```

Checking Arithmetic

Checking propagates the expected type to both operands.

```
tc_check( $\Gamma f$ ,  $\Gamma v$ ,  $e1 + e2$ ,  $\tau$ ):  
  require  $\tau \in \{i32, \text{usize}\}$   
   $t1 = \text{tc\_check}(\Gamma f, \Gamma v, e1, \tau)$   
   $t2 = \text{tc\_check}(\Gamma f, \Gamma v, e2, \tau)$   
  return  $(t1 + t2 : \tau)$ 
```

```
let n: usize = 2;  
let x: usize = n + 1; // accepted  
let y = n + 1;        // rejected: usize + i32
```

Synthesis infers both operands independently and requires the same numeric type.
The same policy applies to $-$, $*$, $/$, $\%$.

Checking Comparisons

A bool result does not determine the numeric operand type.

```
tc_check( $\Gamma f$ ,  $\Gamma v$ ,  $e1 < e2$ , bool):  
    t1 = tc_synth( $\Gamma f$ ,  $\Gamma v$ , e1)  
    t2 = tc_synth( $\Gamma f$ ,  $\Gamma v$ , e2)  
    require type_of(t1) = type_of(t2)  $\in$  {i32, usize}  
    return (t1 < t2 : bool)  
  
let n: usize = 2;  
let b: bool = n < 1;           // rejected: usize vs. i32  
let k: usize = 1;  
let c: bool = n < k;           // accepted  
RustY uses this policy for <, >, <=, >=, ==, !=.
```

This implementation does not support equality on bool.

Checking Arrays

Checking $[e_1, \dots, e_n]$ against $[\tau; k]$

Require $n = k$, then check each e_i against τ .

```
let a: [usize; 2] = [1, 2 + 3];  
// Each element and each numeric literal has type usize.
```

Synthesis requires a nonempty array with matching element types.

```
let b = [1, 2];           // [i32; 2]  
let c = [];               // cannot infer the element type  
let d: [i32; 0] = [];     // accepted  
let e: [i32; 3] = [1, 2]; // rejected: length mismatch
```

Array Indexing

Synthesize the array type, then check the index against `usize`.

```
tc_synth( $\Gamma f$ ,  $\Gamma v$ , a[i]):  
    ta = tc_synth( $\Gamma f$ ,  $\Gamma v$ , a)  
    require type_of(ta) = [ $\tau$ ; n]  
    ti = tc_check( $\Gamma f$ ,  $\Gamma v$ , i, usize)  
    return (ta[ti] :  $\tau$ )  
  
let a = [10, 20];  
let x = a[0];           // accepted: index 0 checks as usize  
let i = 0;              // i : i32  
let y = a[i];           // rejected: i32 index
```

`tc_check` may just use the “fallback” mechanism.

Type checking here does not prove that the index is in bounds.

Function Calls

$$\Gamma f(f) = [\rho_1, \dots, \rho_n] \rightarrow \tau$$

Look up f , check the argument count, and check each argument against its parameter type. Annotate the call with τ .

```
fn f(x: i32, n: usize) -> usize { return n; }
```

```
let y = f(3, 4);    // 3 : i32, 4 : usize, f(3,4) & y : usize
```

Reference parameters need additional checks, covered later.

Checking Statements, Functions, and Programs

Construction of Function Environment

- Globally scan the whole program to construct the global function environment.

```
fn fib(n : i32) -> i32 {  
  ...  
}
```

```
fn eadd(a1: &mut [i32; 3], a2: &[i32; 3]) {  
  ...  
}
```

```
fn main() {  
  ...  
}
```

<pre>Γf = { fib : [i32] -> i32; eadd : [&mut [i32; 3], &[i32; 3]] -> unit; main : [] -> unit }</pre>

Construction of Variable Environment

- For each function, we traverse its body and keep updating the variable environment.

```
Γf = { fib : [i32] -> i32;  
      eadd : [&mut [i32; 3], &[i32; 3]] -> unit;  
      main : [] -> unit }
```

```
fn fib(n : i32) -> i32 {  
  let x = n - 1;  
  ... :i32 ...  
  let y = fib(x);  
    :i32 ...  
}
```

```
Γv = { n : i32 }
```

```
Γv = { x : i32; n : i32 }
```

```
Γv = { y : i32; x : i32; n : i32 }
```

Variable Information

The variable environment stores type and binding *mutability*.

$$\Gamma_v(x) = (\rho, m)$$

ρ is the parameter type. m says whether we can assign to x .

<code>fn f(n: i32, p: &mut i32) {</code>	<code>n : (i32, false)</code>
<code> let x = n;</code>	<code>p : (&mut i32, false)</code>
<code> let mut y = n;</code>	<code>x : (i32, false)</code>
<code>}</code>	<code>y : (i32, true)</code>

All parameter bindings are immutable.

An immutable reference binding can still allow writing through `*p`.

Checking Statements

$\text{tc_stmt}(\tau_r, \Gamma_f, \Gamma_v, s) : \text{venv} \times \text{tstmt}$

$\text{tc_block}(\tau_r, \Gamma_f, \Gamma_v, b) : \text{tblock}$

τ_r is the enclosing function's return type.

A statement produces a typed statement and an environment.

A declaration adds a binding for later statements.

$$\begin{array}{ccccccc} \Gamma_v^0 & \rightarrow & \Gamma_v^1 & \rightarrow & \Gamma_v^2 & \rightarrow & \dots \rightarrow \Gamma_v^n \\ s_1 & & s_2 & & & & s_n \end{array}$$

A block checks statements in order and keeps local bindings inside.

Variable Declarations

```
let x = e;  
  te = tc_synth( $\Gamma f$ ,  $\Gamma v$ , e)  
   $\Gamma v'$  =  $\Gamma v[x \mapsto (\text{type\_of}(te), \text{false})]$ 
```

```
let x:  $\tau$  = e;  
  te = tc_check( $\Gamma f$ ,  $\Gamma v$ , e,  $\tau$ )  
   $\Gamma v'$  =  $\Gamma v[x \mapsto (\tau, \text{false})]$ 
```

For let mut, record true instead of false.

Check the initializer before adding the new binding.

```
let x: i32 = x + 1;    // the initializer uses the previous x
```

Assignments

```
l = r;  
  require l to be a mutable location  
  t1 = tc_synth( $\Gamma$ f,  $\Gamma$ v, l)  
  tr = tc_check( $\Gamma$ f,  $\Gamma$ v, r, type_of(t1))
```

Mutable locations

```
x      if its binding is mutable  
*p     if p has type &mut  $\tau$   
a[i]   if a is a mutable location
```

```
let mut n: usize = 0;  
n = n + 1; // accepted      n = true; // rejected
```

Assignment leaves the variable environment unchanged.

Conditionals and Loops

Check the condition against bool & check the blocks

```
if condition { b1 } else { b2 }  
  tc_check( $\Gamma f$ ,  $\Gamma v$ , condition, bool)  
  tc_block( $\tau r$ ,  $\Gamma f$ ,  $\Gamma v$ , b1)  
  tc_block( $\tau r$ ,  $\Gamma f$ ,  $\Gamma v$ , b2)
```

```
while condition { b }  
  tc_check( $\Gamma f$ ,  $\Gamma v$ , condition, bool)  
  tc_block( $\tau r$ ,  $\Gamma f$ ,  $\Gamma v$ , b)
```

Each block starts with the same outer environment.

(I.e., Its declarations do not extend the outer environment.)

Blocks and Shadowing

```
let x = 3;  
if true {  
    let x = false;  
    let y = 4;  
}  
println!("{}", x);    // x : i32
```

Before: [x : i32]

Inside: [y : i32; x : bool; x : i32]

After: [x : i32]

Lookup finds the most recent binding first.

The inner x and y disappear when we leave the block.

Returns and Other Statements

Let τ_r be the function's declared return type.

```
return e;  
    tc_check( $\Gamma_f$ ,  $\Gamma_v$ , e,  $\tau_r$ )  
return;  
    require  $\tau_r = \text{unit}$   
  
e;  
    tc_synth( $\Gamma_f$ ,  $\Gamma_v$ , e)  
println!("{}", e);  
    synthesize e and require i32 or usize
```

These statements leave Γ_v unchanged.

This checker does not verify that every control-flow path returns.

Checking Functions and Programs

Program

1. Reject duplicate function names.
2. Collect every signature into Γ_f .
3. Check each function using Γ_f .

Each function

1. Reject duplicate parameter names.
2. Initialize Γ_v from the immutable parameter bindings.
3. Check the body using its declared return type τ_r .

All signatures are available before checking any body.

Forward calls, recursion, and mutual recursion are possible.

Borrow Checking at Function Calls

References in RustY

References appear in parameter types and call arguments.

```
fn update(p: &mut i32, q: &i32) {  
    *p = *p + *q;    // accepted  
    // *q = 0;      // would be rejected  
}
```

Both $\&\tau$ and $\&\text{mut } \tau$ allow reading through $*p$.

Only $\&\text{mut } \tau$ allows writing through $*p$.

The parameter binding itself remains immutable.

Use a reference parameter by dereferencing it or forwarding it as a reference argument. RustY has no local reference bindings.

Creating Reference Arguments

Arguments are checked against their parameter types:

Parameter τ	e	check e against τ
Parameter $\&\tau$	$\&x$	require $x : \tau$
Parameter $\&\text{mut } \tau$	$\&\text{mut } x$	also require a mutable binding
Parameter $\&\tau/\&\text{mut } \tau$	x	forwarding

```
fn add_to(p: &mut i32, delta: i32) {  
    *p = *p + delta;  
}  
let mut n = 0;           // { n : (i32, true), .. }  
add_to(&mut n, 2);
```

Borrow Checking Within a Call

Track borrows of each variable during one call.

$B(x) \in \{\text{shared}, \text{exclusive}\}$ Initially, $B = \emptyset$

Existing borrow	New shared	New exclusive
none	allow	allow
shared	allow	reject
exclusive	reject	reject

```
inspect(&x, &x);           // accepted  
update(&mut x, &x);       // rejected
```

inspect takes `[&i32, &i32]`. update takes `[&mut i32, &i32]`.

Assume `x` is a mutable `i32` variable.

Forwarding Reference Parameters

Pass an existing reference by its variable name.

Existing parameter	Expected $\&\tau$	Expected $\&\text{mut } \tau$
$p : \&\tau$	allow	reject
$p : \&\text{mut } \tau$	allow	allow

```
fn twice(p: &mut i32) {  
    add_to(p, 1);  
    add_to(p, 1);  
}
```

Forwarding produces `TAE_fwd p`.

Conflicting borrows within the same call still fail.

Putting It Together

```
fn add_to(p: &mut i32, delta: i32) {  
    *p = *p + delta;  
}
```

```
fn main() {  
    let mut n = 0;  
    while n < 6 {  
        add_to(&mut n, 2);  
    }  
    println!("{}", n);  
}
```

*p is a mutable i32 location.

&mut n matches &mut i32 without a conflicting borrow.

Summary: AST to Typed AST

$\text{tc_prog} : \text{prog} \rightarrow \text{tprog}$ (The last assignment before midterm.)

- Collect function signatures before checking bodies.
- Track local types, binding mutability, and block scope.
- Use synthesis and checking to annotate each expression.
- Check assignments, conditions, and returns.
- Check reference arguments and borrows within each call.

IR generation uses the typed AST to choose LLVM types, instructions, and memory operations.